МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»
ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ

Лабораторна робота №4
з дисципліни “Теорія інформації і кодування”

на тему: “ ДОСЛІДЖЕННЯ ВЛАСТИВОСТЕЙ ТА ОСНОВНИХ ХАРАКТЕРИСТИК КОДІВ ХЕММІНГА”

Виконав:
студент III курсу
групи ДА-11
Сидько Нікіта Сергійович

Київ-2023

Мета роботи – дослідження алгоритмів побудови кодів Хеммінга та їх можливостей виявлення та корекції помилок при використанні в інформаційних технологіях.

Завдання

1. Ознайомитись з принципами побудови та основними характеристиками кодів Хемммінга.

2. Розробити алгоритми **кодування** текстових повідомлень з використанням коду Хемммінга з мінімальною кодовою відстанню dmin =3 і dmin =4.

3. Розробити алгоритми **декодування** текстових повідомлень з використанням коду Хемммінга з мінімальною кодовою відстанню dmin =3 і dmin =4.

4. Реалізувати розроблені алгоритми на мові програмування високого рівня. В програмах передбачити можливість вводу текстових повідомлень вручну з клавіатури, а також з підготовлених заздалегідь файлів.

5. Підготувати текстовий файл з наступним змістом:

- Своє прізвище, ім'я, по батькові, рік та місце народження, група, який навчальний заклад закінчив та ін.
- Зберегти файл у форматі з підримкою кодової таблиці Windows 1251.

6. Перевірити коректність роботи розроблених кодерів і декодерів для кодів Хемммінга (п.4 та п.5) без внесення помилок.

7. Модифікувати програми кодерів і декодерів для забезпечення можливості внесення вручну однократних та багатократних помилок в текстові повідомлення. Провести тестування декодерів при внесенні однократних помилок в текстові повідомлення.

8. Провести тестування кодерів і декодерів при внесенні вручну багатократних помилок (2, 3, 4, 5 ….) в текстові повідомлення .Результати тестування задокументувати.

9. Модифікувати програми кодерів і декодерів для забезпечення можливості автоматичного внесення однократних та багатократних помилок в файли з текстовими повідомленнями. Кількість помилок повинна вводитись в діалоговому режимі. Провести тестування на файлах довільної довжини і кількості помилок.

Коди Хеммінга - одні з найпоширеніших систематичних кодів, які виправляють помилки. До кодів Хеммінга належать коди з мінімальною відстанню $d_{mjn} = 3$, що виправляють всі поодинокі помилки. Формування ***r*** **перевірних елементів** у комбінаціях цих кодів обчислюють за ***k*** **інформаційними елементами**. Таким чином, довжина кодової комбінації $n = k + r$. Перевірними елементами є лінійні комбінації інформаційних елементів, тобто зважені суми інформаційних елементів з ваговими коефіцієнтами 1 та 0.

Кодування для dmin = 3.

1. Створимо масив бітів розміру M = N + R, де N - довжина вихідного повідомлення, R - число контрольних розрядів (яке задовольняє 1).
2. Розрахуємо значення контрольних розрядів, для створеного масиву бітів.
3. Перенесемо інформаційні біти в створений масив.

Кодування для dmin = 4.

1. Створимо масив бітів розміру M = N + R + 1, де N - довжина вихідного повідомлення, R - число контрольних розрядів (яке задовольняє 1).
2. Розрахуємо значення контрольних розрядів та помістимо їх в створений масив.
3. Перенесемо інформаційні біти в створений масив.
4. Додаємо ще один контрольний біт в початок, який визначає парність усього коду. Якщо сума по модулю усіх біт = 0, то цей контрольний розряд набуває значення 0 та навпаки.

Декодування dmin = 3.

1. Перерахуємо значення контрольних розрядів. Якщо обраховане значення співпадає зі значеннями закодованими в повідомленні, то повідомлення вірне.
2. Якщо значення відрізняються, можемо локалізувати помилку, додавши значення контрольних бітів, які відрізняються, це і буде індексом шуканого елементу.
3. Якщо був знайдений пошкоджений біт, інвертуємо його значення.

Декодування dmin = 4.

1. Перерахуємо значення контрольних розрядів. Якщо обраховане значення співпадає зі значеннями закодованими в повідомленні, то повідомлення вірне.
2. Перевіримо парність всього коду, крім нульового контрольного розряду. Якщо співпадають, то маємо дві помилки.
3. Якщо не співпадає, то маємо 1 помилку, яку можемо виправити.

Реалізація на Python

```
from bitarray import bitarray
from functools import reduce
from random import randint

def encode_file(input_file_name, num_errors):
    with open(input_file_name, 'rb') as input_file:
        with open(f'{input_file_name}d.txt', 'wb') as
output_file:
            while True:
                input_bits = bitarray()
                raw_bytes = input_file.read(15)
                if not raw_bytes:
                    break
                input_bits.frombytes(raw_bytes)
                encoded_bits = hamming_encode(input_bits)
                errored_bits = introduce_errors(encoded_bits,
num_errors)
                errored_bits.tofile(output_file)

def hamming_encode(bits):
    if len(bits) < 120:
        return bits
    encoded_bits = encode(bits, 7)
    encoded_bits.insert(0, reduce(lambda x, y: x ^ y,
encoded_bits))
    return encoded_bits

def encode(bits, max_power):
    for i in range(max_power):
        bits.insert(2 ** i - 1, 0)
    step = 2
    for i in [2 ** j - 1 for j in range(0, max_power)]:
        parity_check_bits = []
        parity_bit_index = i
        while i < len(bits):
            parity_check_bits.extend(val for val in bits[i:i +
step // 2])
            i += step
        parity = reduce(lambda x, y: x ^ y, parity_check_bits)
        bits[parity_bit_index] = parity
        step *= 2
    return bits

def introduce_errors(bits, num_errors):
    if num_errors == 1:
        error_index = randint(0, len(bits) - 1)
        bits[error_index] = not bits[error_index]
    elif num_errors >= 2:
        error_indices = set()
        for _ in range(num_errors):
            while True:
```

```
                error_index = randint(0, len(bits) - 1)
                if error_index not in error_indices:
                    error_indices.add(error_index)
                    bits[error_index] = not bits[error_index]
                    break
    return bits

def decode_file(input_file_name, output_file_name):
    with open(f'{input_file_name}d.txt', 'rb') as input_file:
        with open(output_file_name, 'wb') as output_file:
            while True:
                bits_from_file = bitarray()
                raw_bytes = input_file.read(16)
                if not raw_bytes:
                    break
                bits_from_file.frombytes(raw_bytes)
                decoded_bits = hamming_decode(bits_from_file)
                if decoded_bits is not None:
                    decoded_bits.tofile(output_file)
                else:
                    break

def decode(bits, max_power):
    step = 2
    error_bit_index = None
    for i in [2 ** j - 1 for j in range(0, max_power)]:
        parity_check_bits = []
        parity_bit_index = i
        while i < len(bits):
            parity_check_bits.extend(val for val in bits[i:i +
step // 2])
            i += step
        parity_check_bits[0] = 0
        parity = reduce(lambda x, y: x ^ y, parity_check_bits)
        if parity != bits[parity_bit_index]:
            if error_bit_index is None:
                error_bit_index = parity_bit_index
            else:
                error_bit_index += parity_bit_index + 1
        step *= 2
    if error_bit_index is not None:
        print(f'The error was found at position:
{error_bit_index}')
        bits[error_bit_index] = not bits[error_bit_index]
    for i in reversed(range(0, max_power)):
        del bits[2 ** i - 1]
    return bits

def main():
    input_file = "text.txt"
    output_file = "out.txt"
    while True:
        print('\n1. Encode\n2. Decode\n3. Encode and decode\n4.
```

```
Exit\n')
        option = input('Choose a function: ')

        if option == '1':
            error_num = int(input("How many mistakes: "))
            encode_file(input_file, error_num)

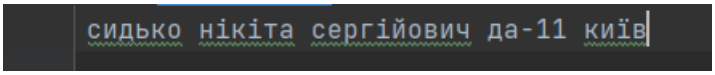
        elif option == '2':
            decode_file(input_file, output_file)

        elif option == "3":
            error_num = int(input("How many mistakes: "))
            encode_file(input_file, error_num)
            decode_file(input_file, output_file)

        elif option == '4':
            break

if __name__ == '__main__':
    main()
```

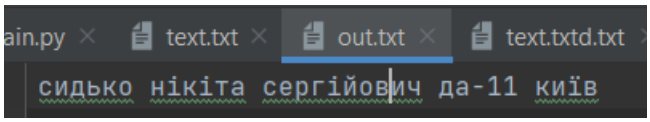
Текстовий файл:



Закодований:



Розкодований :



Без внесення помилок.

```
Choose a function: 3
How many mistakes: 0
There are no errors.
```

З помилками

```
Choose a function: 3
How many mistakes: 1
The error was found at position: 25
```

Висновок

У ході виконання лабораторної роботи було розглянуто побудову та застосування контрольованих і керованих кодів Хемінга, приктично розв’язано приклади та розрахунки.